



Karmada 与 Xline：跨集群管理有状态应用的探索与实践

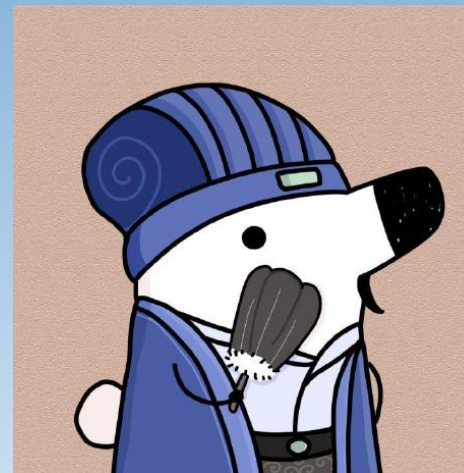


赵佳炜

- Xline 社区 Maintainer
- Datenlord 存储研发工程师
- 开源信徒

github: [Phoenix500526](https://github.com/Phoenix500526)

blog: xline.cloud





Content 目录

01 背景与动机

02 挑战与现状

03 早期阶段的探索与尝试

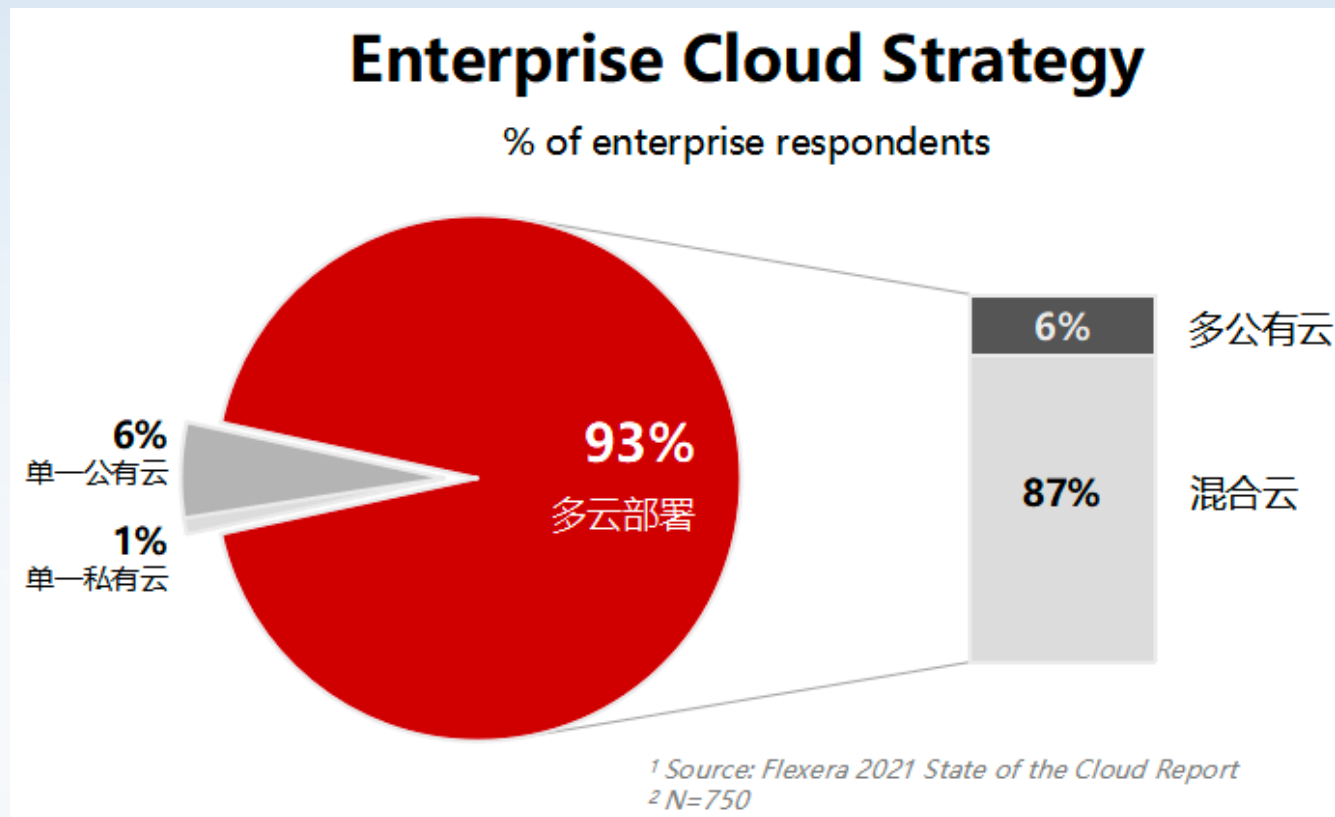
04 总结与展望



Part 01

背景与动机

外部因素



调查显示，超过 93% 的企业正同时使用多个云厂商的服务
云原生技术和云市场不断成熟，未来将会是程式多云管理服务的时代

内部因素

- 单集群本身的边界限制，一个 Node 默认只有 110 个 pod，一个集群最多容纳 5000 个 Node
- 业务本身的发展，例如 Xline 本身做为跨云的分布式 KV 存储，单集群内不能充分发挥其优势
- 故障域的容忍，多集群能够更好的容忍集群级别的故障
-



Part 02

现状与挑战

Karmada 生产应用的现状

组织/公司名称	使用场景	使用案例
DaoCloud	构建更灵活，更有弹性的混合多云平台	DaoCloud结合Karmada打造新一代企业级多云平台
飓风引擎	零门槛低代码平台支持用户零代码快速构建自己的应用，并通过 karmada 一键发布到任意平台	艾莫尔研究院基于 Karmada 的落地实践
Shopee	实现多集群管理与应用分发	Karmada 在 Shopee 的落地实践
VIPKID	搭建多厂商多地域的多云 PaaS 平台	使用 Karmada 构建运行容器的 PaaS 平台 —— VIPKID

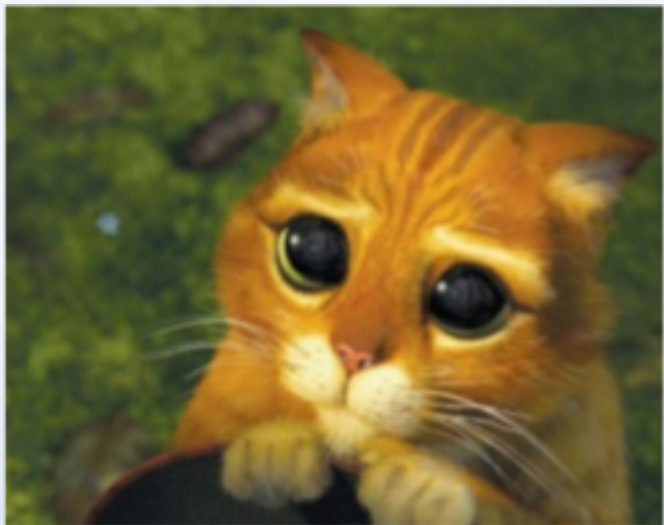
更多生产实例可参考：<https://karmada.io/zh/adopters/>

Cattles VS Pets



Cattles (无状态)

- 不需要有特定的名字
- 个体之间没有本质区别
- 当它们生病时，直接用另一个个体来代替



Pets (有状态)

- 有特定的名字 (标识符)
- 独一无二的，需要特殊的照顾
- 它们可能会生病，需要帮助它们恢复健康

什么是状态？

单集群下一个有状态应用都有哪些状态。

- 应用状态持久化：例如状态机日志，数据表等；
- 固定的网络标志：client 能够始终以相同的标识访问到同一个实例上，而无论这个实例是否被重启过；
- 拓扑关系：包括实例依赖关系，以及启动顺序的要求；

K8s 是如何通过 StatefulSet 管理有状态应用

K8s 在 1.5 版本中首次引入了 StatefulSet，并在 1.9 版本中稳定可用的状态。目前已经被广泛应用于运行有状态应用。它为所管理的 Pod 提供了固定的 Pod 身份标识，每个 Pod 的持久化存储以及 Pod 之间严格的启停顺序

- 通过与固定 Pod 身份标识同名的 PVC，解决了应用状态持久化的问题；
- 通过固定的 Pod 身份标识与 Headless Service，提供了不变的 dns name，解决了固定的网络标识问题；
- 通过 Pod 之间严格的启停顺序，解决了有状态应用拓扑关系的问题；

跨集群状态下，应用的状态如何保证？

- 如何保证跨集群中应用实例有全局统一的启动顺序
- 如何保证跨集群的所有应用实例都有全局唯一的实例标识
- 如何解决跨集群的应用通信以及提供全局统一的网络标识
- 如何解决跨集群的有状态应用的更新、扩缩容等相关功能
-

上述问题的解决都需要新的 karmada API 的实现，实现跨集群的 StatefulSet



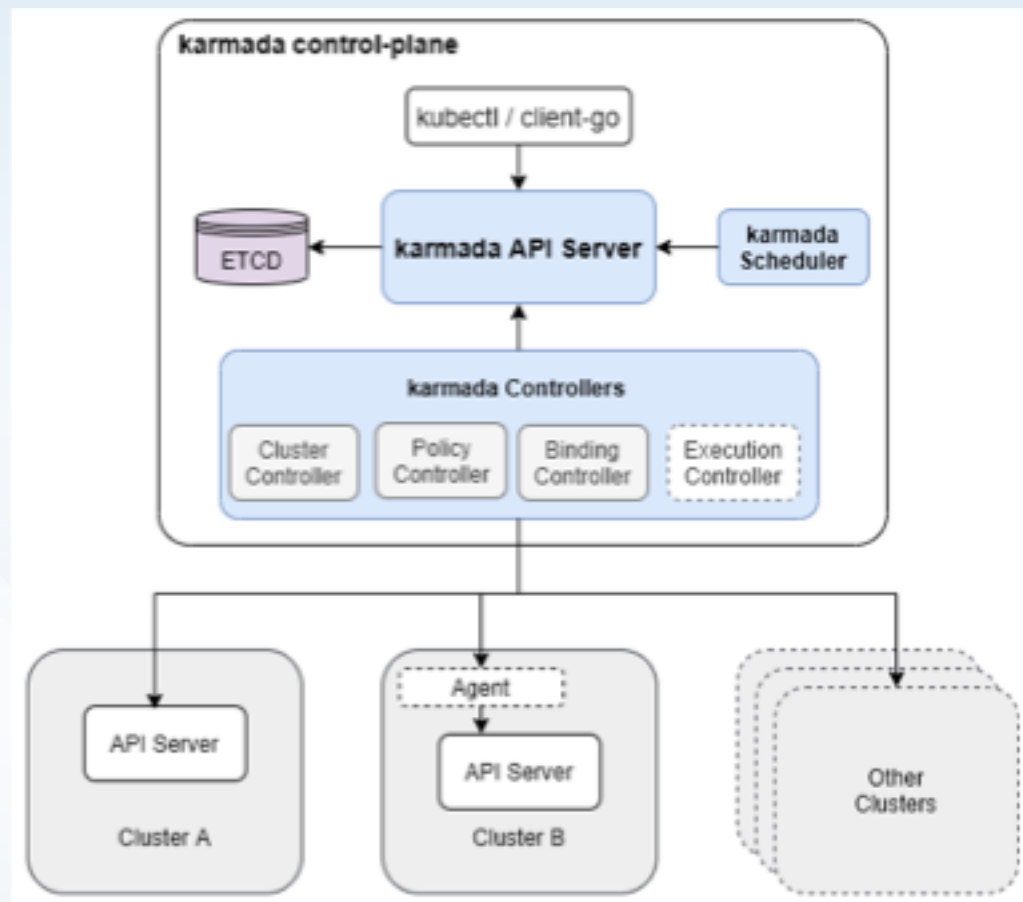
Part 03

早期阶段的探索与尝试

Karmada 简介

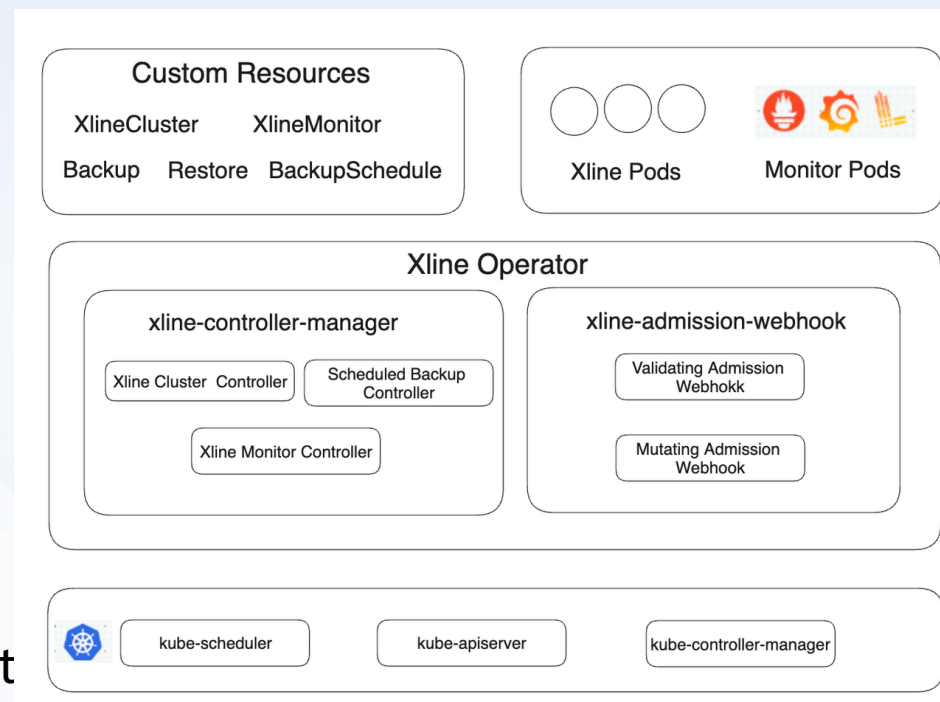
- Karmada 是一个 K8s 管理系统，能够让用户在多个 K8s 集群和云中运行云原生应用程序，而无需更改应用
- 兼容 K8s 原生 API
- 集中式管理
- CNCF sandbox project

GitHub Repo: <https://github.com/karmada-io/karmada>



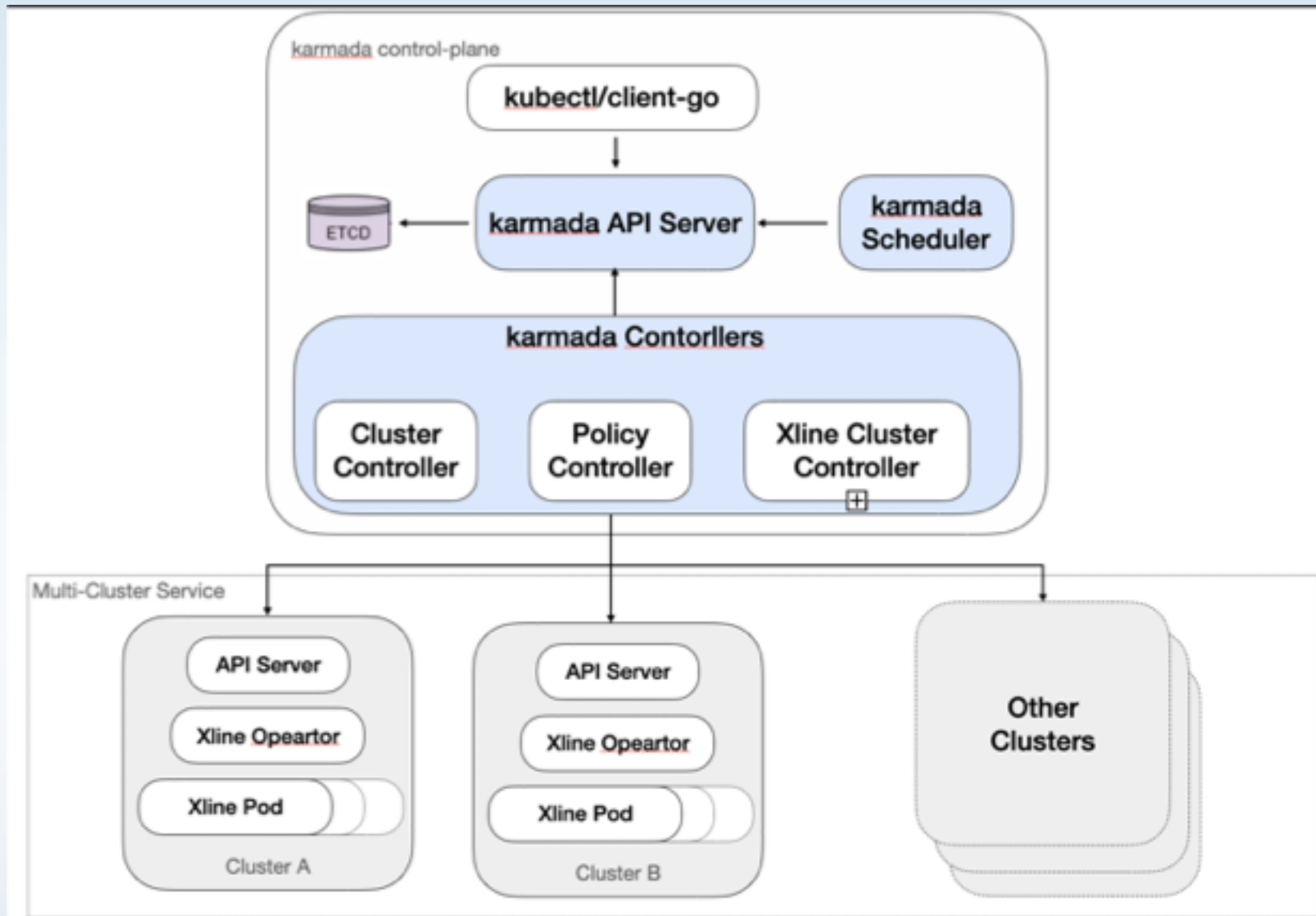
Xline Operator

- 用来管理单个 K8s 集群下的 Xline Cluster
- Xline 是一个立足跨云场景的分布式 KV 存储，采用 CURP 协议作为后端共识协议，目前也是 CNCF Sandbox Project



GitHub Repo: <https://github.com/xline-kv/xline-operator>

Architecture



单集群下分布式应用集群如何部署

单集群部署分布式应用集群的常见做法

Etcd Operator :

1. Bootstrap: 创建一个 etcd 的种子节点，种子节点的 initial-cluster-state 为 new，并制定了唯一的 initial-cluster-token
2. Scale out : 在种子集群上执行 member add，更新集群网络拓扑，然后启动新的 etcd 节点，新节点中的 initial-cluster 为更新后的网络拓扑，并且 initial-cluster-state 为 existing

多集群场景下部署的一些问题

在单集群的场景下，使用动态扩充的方式部署集群有以下特点：

1. 部署和扩缩容的逻辑是一致的；
2. 部署逻辑符合 Reconcile 语义；
3. 单集群下的启动完了一个 pod 再启动下一个 pod，满足单节点变更的语义

多集群场景下，动态部署带来的一些问题：

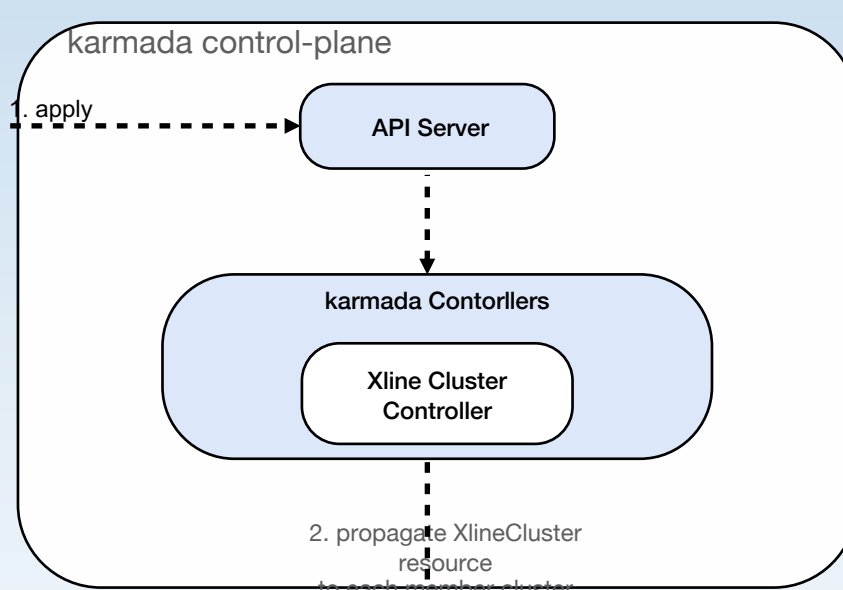
1. 多个 member cluster 之间启动 pod 的操作缺少同步机制，对于使用单节点变更的 Xline 而言会带来正确性的问题
2. 动态部署依赖于集群共识，member cluster crash 可能会导致 majority 被破坏，其他健康的 member cluster 无法继续部署新节点。

Xline 如何部署到 Karmada 多集群中

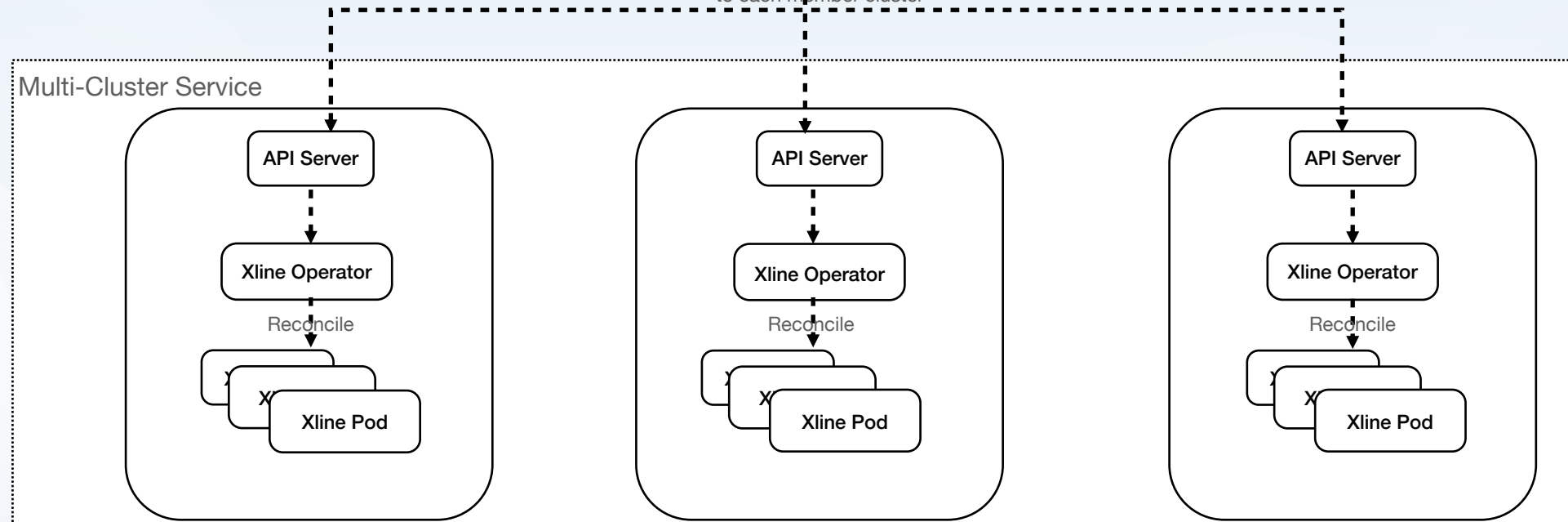
前置条件：

1. 部署好 member cluster 并 join 到 karmada 中
2. 在 member cluster 当中部署好 xline-operator
3. 在 member cluster 当中创建好 multi-cluster service，以便不同 member cluster 之间可以彼此互通

karmada resource schema:
....
version: vx.y.z
Image: ghcr.io/xline-kv/xline
name: xline-cluster
member cluster:
 member1: 3
 member2: 3
 member3: 3
....



XlineCluster resource schema:
...
kind: XlineCluster
metadata:
 name: xline-cluster
spec:
 version: vx.y.z
 Image: ghcr.io/xline-kv/xline
 replicas: 3
...



Rolling Update

....
version: vx.y.y

Image: ghcr.io/xline-kv/xline

name: xline-cluster

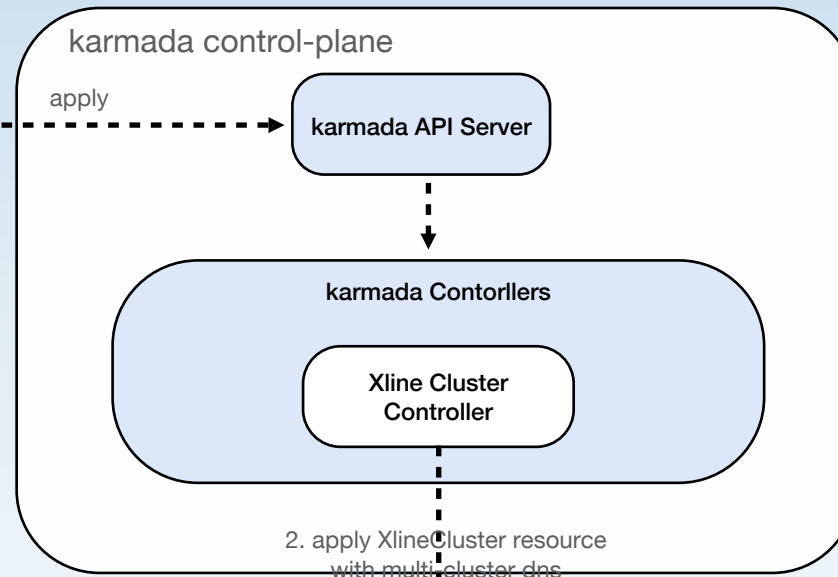
member cluster:

member1: 3

member2: 3

member3: 3

...



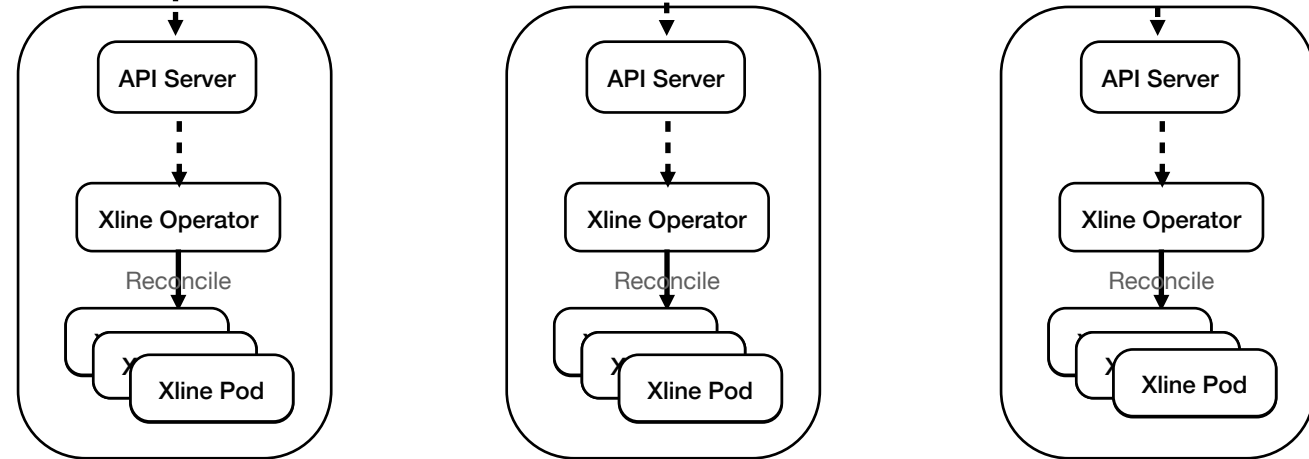
...
kind: XlineCluster
metadata:
name: xline-cluster
spec:

version: vx.y.y

Image: ghcr.io/xline-kv/xline

replicas: 4
...

Multi-Cluster Service



Karmada 多集群下如何 scale out

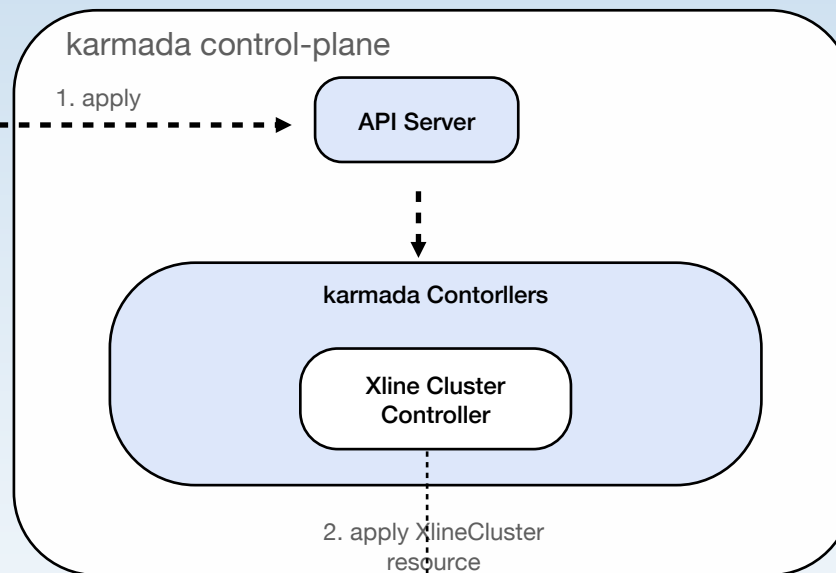
Karmada 下的 scale out 可以分为两种：

- 水平 scale out —— 增加一个 member cluster，并在其上 scale out 节点
- 垂直 scale out —— 在原有的 member cluster 上进行 scale out

水平 scale out

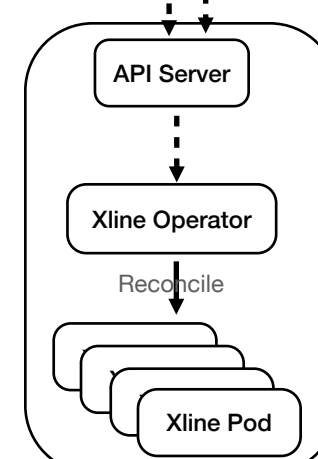
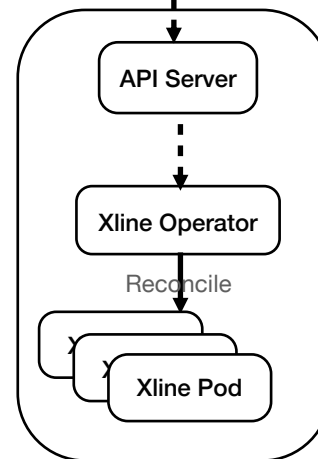
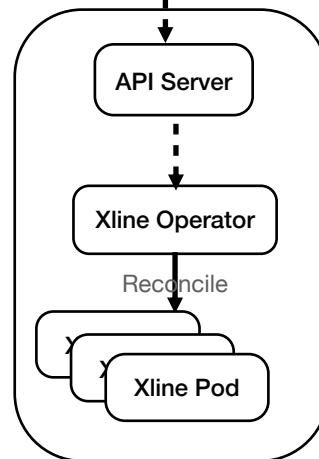
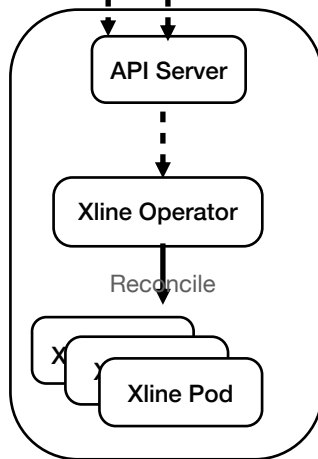
....
version: vx.y.z
Image: ghcr.io/xline-kv/xline
name: xline-cluster
member cluster:
 member1: 3
 member2: 3
 member3: 3
 member4: 4
....

...
kind: XlineCluster
metadata:
 name: xline-cluster
spec:
 version: vx.y.z
 Image: ghcr.io/xline-kv/xline
 replicas: 3
...



...
kind: XlineCluster
metadata:
 name: xline-cluster
spec:
 version: vx.y.z
 Image: ghcr.io/xline-kv/xline
 replicas: 4
...

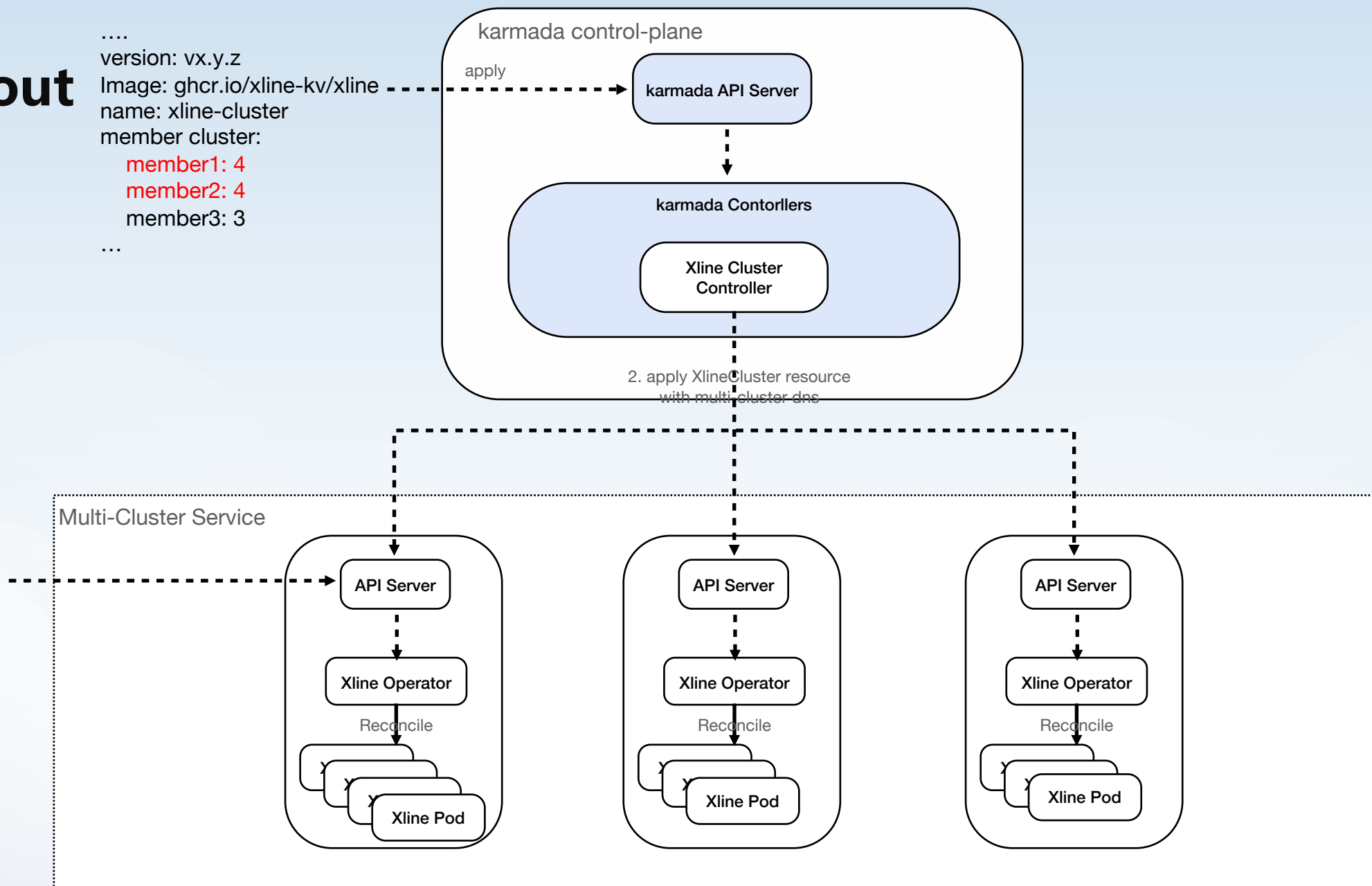
Multi-Cluster Service



垂直 scale out

....
version: vx.y.z
Image: ghcr.io/xline-kv/xline
name: xline-cluster
member cluster:
 member1: 4
 member2: 4
 member3: 3
....

...
kind: XlineCluster
metadata:
 name: xline-cluster
spec:
 version: vx.y.z
 Image: ghcr.io/xline-kv/xline
 replicas: 4
...





Part 04

总结与展望

总结

- 目前 Xline 社区和 Karmada 社区成立了工作小组，共同推动有状态应用在 Karmada 多集群下的管理
- 目前 Karmada 社区关于 StatefulSet workload 的一些实现细节还没有达成共识，因此在早期阶段，Xline 采用了两层的 Operator 的方式，对部署、更新等问题做了些探索和尝试，为未来的 Karmada StatefulSet workload 的开发与设计做前期的铺垫

未来的挑战

- 如何在 Karmada 上感知到成员集群的资源变化，从而更好地支持有状态应用的部署
- 如何在 Karmada 上实现跨集群的有状态应用的可观测性与监控
- 如何支持更加细粒度更新策略，例如 Partition Update
- 如何对跨集群网络做抽象，使其不依赖于某种具体的网络插件，如 submariner
- 如何降低把单集群下有状态应用的 Operator 迁移到 Karmada 多集群场景下的迁移成本
-



Thanks.

